CFC Informaticien

Travail Pratique Individuel (P_APPRO1)

P_Noveling

Candidat	*Julien Mares*
École professionnelle / Entreprise formatrice	ETML
Chef de projet	*Mathieu Meylan*
Expert 1	
Expert 2	
Date de début	*02.02.2026*
Date de fin	*13.03.2026*
Version du rapport	1.0.0

Table des matières

1 Partie 1 - Rapport de projet 6

1.1 Contexte et mandat 6

1.1.1 Mandat et parties prenants. 6

1.1.2 Périmètre (in-scope / out-of-scope) 6

1.1.3 Livrables attendus (inclure versions/liaisons dépôt) 6

1.2 Méthode et organisation 6

1.2.1 Méthode de gestion de projet 6

1.2.2 Mise en place d'une ToDo List Trello (Planification Agile) 7

1.2.3 Organisation (rôles, responsabilités, communication) 7

1.2.4 Outils (gestion de versions, suivi tâches,) 7

1.3 Analyse des exigences 7

1.3.1 Exigences fonctionnelles et non fonctionnelles (qualité, performance, sécurité, UX) 7

1.4 Gestion des risques et mesures 9

1.4.1 Risques techniques 9

1.4.2 Risques organisationnels 9

1.5 Conception 9

1.5.1 Maquette UI 9

1.5.2 Analyse fonctionnelle 10

1.5.3 Choix techniques et alternatives évaluées 12

1.5.4 Sécurité : authentification, autorisations, gestion des secrets 12

1.6 Planification et suivi 13

1.6.1 Planification initiale (jalons, Gantt, tâches, date de début / fin, Milestone) 13

1.6.2 Planification révisée et écarts (explications causes + actions) 13

1.7 Réalisation 14

1.7.1 Description de l'implémentation (composants, modules, conventions) 14

1.8 Tests et validation 17

1.8.1 Tests fonctionnels 17

1.8.2 Test unitaires (Pas mis en place) 22

1.9 Conclusion et bilan personnel 22

1.9.1 Objectifs atteints / non atteints (preuve) 22

1.9.2 Points positifs / négatifs 22

1.9.3 Difficultés rencontrées et solutions 22

1.9.4 Apprentissages et perspectives d'amélioration 23

2 Partie 2 - Documentation du produit 24

2.1 Installation du projet (environnement développement) 24

2.1.1 Environnement de base et applications de base requis 24

2.1.2 Installation du projet 24

2.2 Guide d'utilisation **25**
2.3 Architecture technique (rappel) **25**
2.3.1 Schéma d'architecture 25
3 Résumé **25**
3.1 Contexte et mandat **25**
3.2 Solution réalisée **25**
3.3 Résultats principaux **25**
3.4 Limites et suites **26**
4 Annexes **27**
A. Journal de travail **27**
B. Sources et références **30**
4.1 Références **30**
4.2 Sources **30**
4.3 Usage IA **31**
C. Liste des livrables remis **31**

Gestion du document

Abréviations et glossaire

Vue	*Une vue est un composant Vue défini dans un fichier .vue qui regroupe la structure (template), la logique (script) et le style (style) d'une partie de l'interface utilisateur.*
Store	*Un store est un espace centralisé de gestion d'état qui permet de stocker et partager les données de l'application entre plusieurs composants.*
Router guard	*Un router guard est une fonction exécutée avant un changement de page afin de vérifier certaines conditions (comme l'authentification) et autoriser ou bloquer l'accès à une route*
Middleware	*Fonction intermédiaire qui s'exécute entre la requête et la réponse.*

1 Partie 1 - Rapport de projet

1.1 Contexte et mandat

1.1.1 Mandat et parties prenants.

Le projet consiste à la création d'un site web où les utilisateurs peuvent emprunter des livres. Le site se devra d'implémenter une couche de sécurité minimal ainsi que des tests permettant d'assurer le bon fonctionnement

1.1.2 Périmètre (in-scope / out-of-scope)

Tout le projet est dans le périmètre étant donné que c'est un nouveau projet, et non un ajout/amélioration d'un autre.

1.1.3 Livrables attendus (inclure versions/liaisons dépôt)

- Code source du projet
- Journal de travail
- Release GitHub
- Lien vers mon Trello

1.2 Méthode et organisation

1.2.1 Méthode de gestion de projet

Une première analyse a été faites pour poser des bases, c'est pour avoir une grande idée fixe du projet, puis dans un scope un plus réduit une méthode de travail agile, où la planification, les méthodes et outils utilisées peuvent être repensé en fonction des expériences rencontrées.

1.2.1.1 Planification initiale (Waterfall)

Avant de commencer quoi que ce soit, une planification initiale a été faites, permettant de clarifier mes idées et de me faire une idée globale du projet.

Tâches à effectuer	Durée estimé
Planification initiale	6h
Mise en place d'une planification + agile (ex. trello)	1h
Me documenter sur Google Books API	1h30
Création MLD	3h
Création charte graphique + Maquettes	8h
Création de la DB (Si dockerisé faire le docker-compose aussi)	2h30
Mise en place architecture des fichiers du code	1h
Me redocumenter / former dans les bases en JS (Express, VueJS, (Tailwind et FlowBit potentiellement, etc.)	4h
Me redocumenter sur comment faire les Tests unitaires, et les faire	6h
Page web « Inscription » & « Connexion » utilisateurs	6h
Page web « Gestion utilisateurs »	6h
Google Books API (Voir comment ça marche en pratique)	3h

Page web « Gestion des livres »	6h
Page web « Accueil »	6h
Page web « Catalogue de Livres »	6h
Page web d'un livre spécifique (pour avoir + d'infos dessus, ou l'emprunter)	8h
Documentation : Squelette de la doc, sommaire	20m
Documentation : Analyse du projet	3h10
Documentation : Résultat obtenu & explication	5h
Documentation : Simplement mettre la charte graphique + maquette (fais sur canva avant sûrement)	30m
Documentation : Partie Tests	3h
Documentation : Conclusions	1h
Faire le readme (doc pour installation du projet par un tierce)	3h

1.2.2 Mise en place d'une ToDo List Trello (Planification Agile)

En plus de la planification initiale permettant de me faire une première idée du projet, une ToDo list a été faite sur Trello pour un suivi des tâches quotidien et une adaptabilité aux nouvelles contraintes et opportunités.

Lien vers le Trello

1.2.3 Organisation (rôles, responsabilités, communication)

Des discussions régulières autour du projet étaient possible au besoin, lors des cours avec M. Meylan (Chef de projet), et des pré-évaluations effectuées afin d'assurer que la direction prise était la bonne.

1.2.4 Outils (gestion de versions, suivi tâches,)

GitHub et Trello (voir Annexes)

1.3 Analyse des exigences

1.3.1 Exigences fonctionnelles et non fonctionnelles (qualité, performance, sécurité, UX)

1.3.1.1 UX

Des maquettes ont été réalisé pour imager une version du site, vous pouvez les retrouver au Conception, cependant ce sont que des maquettes et il se peut que certaines pages n'y ressemblent finalement pas tant que ça. Je les ai utilisés pour avoir une idée globale au début.

1.3.1.2 Sécurité

Un fichier .env non versionné sur github est utilisé pour les données sensibles (secrets)

1.3.1.3 Fonctionnalités

1.3.1.3.1 Page d'inscription

Une page d'inscription où l'on peut se créer un compte (Email, pseudo, Mot de passe).
Elle permet aussi une redirection vers la page de connexion dans le cas où l'on possède déjà un compte.

1.3.1.3.2 Page de connexion

Une page de connexion où l'on peut se connecter via nos identifiants (Email et mot de passe), cette page nous renvoie un token qui permet de nous authentifier auprès du site et de l'API.

1.3.1.3.3 Page d'accueil

Afficher une liste des 5 livres les plus populaires et une autre des 5 livres les plus récemment ajoutés (titre, couverture, auteur, genre, niveau de popularité, statut d'emprunt).

1.3.1.3.4 Page Catalogue

Afficher une liste complète de tout les livres avec les filtres de recherche suivant (possibilité d'utiliser plusieurs filtres simultanément)

- **Auteur** [Checkbox avec les auteurs, + écoute du clavier pour permettre une recherche d'auteurs dans la liste d'auteurs à cocher]
- **Catégorie** [Checkbox avec les auteurs, + écoute du clavier pour permettre une recherche d'auteurs dans la liste d'auteurs à cocher]
- **Statut d'emprunt**
- **Niveau de popularité** [Possibilité de trier (et non filtrer) par popularité, je pense que le besoin de filtrer par un niveau de popularité est très faible, par-contre cela peut-être intéressant de trier du plus au moins populaire]

La liste a un système de pagination (Permettant un affichage d'une partie de la liste sans avoir un scrollé la page web.

1.3.1.3.5 Page détails livre

Permettre l'affichage de plus d'informations concernant le livre, comme sa préface, ses auteurs, ses genres, sa popularité, etc.

Permettre en tant qu'utilisateur authentifié d'emprunter ce livre pour une certaine durée.

Cette page permet aussi l'ajout d'un commentaire personnel, et l'affichage des commentaires d'autres utilisateurs.

1.3.1.3.6 Page gestion des livres (Admin only)

L'administrateur peut ajouter, supprimer (Uniquement les livres qui ne sont pas empruntés actuellement), modifier les livres de la bibliothèque.
Les livres à ajouter proviennent de l'API Google Books.

1.3.1.3.7 Page gestion des utilisateurs (Admin only)

L'administrateur peut gérer les utilisateurs (modifier, supprimer).

1.3.1.3.8 Page gestions statuts/emprunts livres

Liste tous les emprunts, avec des paramètres possibles

- Emprunt actuel ou fini

- Par utilisateurs

Toutes les informations persistantes du site seront stockées dans une base de données

Le site permettra un affichage responsive sur tablette.

1.3.1.3.9 Sécurité

Les routes de l'API devront être sécurisé, et des permissions accorder selon le statut de l'utilisateur

1.4 Gestion des risques et mesures

1.4.1 Risques techniques

L'un des principaux risques et inconnues techniques concerne l'intégration de l'API Google Books pour ajouter des livres au catalogue. Car, je n'ai encore jamais travaillé avec une API externe dans un projet backend. Mes expériences avec des API externes se limitent jusqu'à présent à simplement récupérer des données, dans le frontend directement, puis les afficher.

Mais il n'y aucune raison que ce soit plus compliqué que cela à priori, je vais simplement me documenter très tôt dans le projet sur la question et ça ne devrait pas poser de problèmes.

1.4.2 Risques organisationnels

Niveau organisationnel, le risque le plus grand que je vois est que vu que cela fait 1 an que je n'ai pas codé/fais de projet du genre, que je sois un peu perdu et que je doive reprendre mes marques, plus qu'un risque, je l'ai déjà remarqué dès les premières semaines du projet.

Aussi le fait de faire du VueJS, un des sujets que je maitrisais le moins l'année passé (selon moi) nécessitera une remise à niveau et compliquera l'analyse initiale et fonctionnel avant d'être à nouveau à l'aise avec les technologies.

En réponse à tout ces risques, je pense que le plus simple est de vite travailler sur la partie frontend (car je suis plus à l'aise avec le backend), afin de pouvoir avoir une vue d'ensemble juste pas trop tardivement.

1.5 Conception

1.5.1 Maquette UI

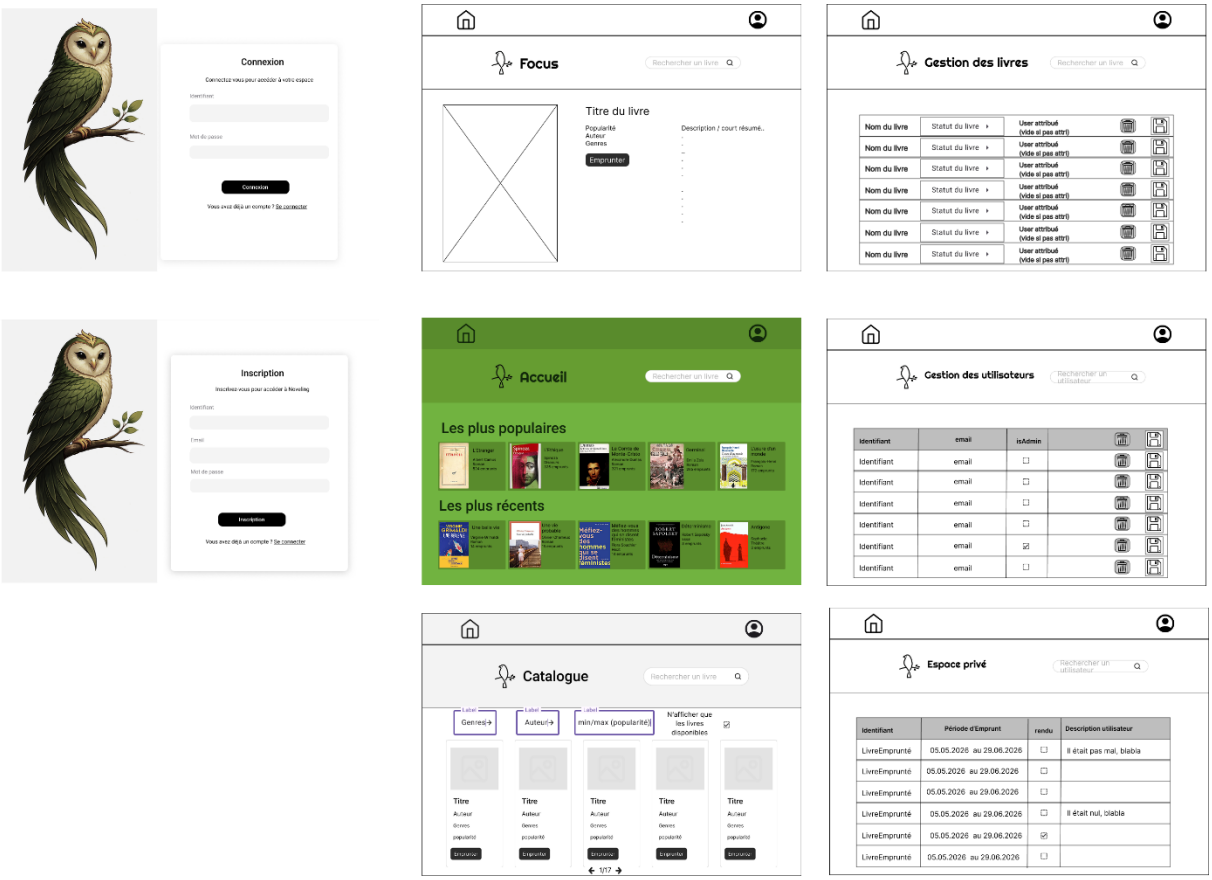
Les maquettes sont disponibles une par une en annexe

1.5.1.1.1 Objectif visuel

Les maquettes ont été faites avec Figma, le résultat recherché est un site simple d'utilisation, où les utilisateurs ne se perdent pas, avec un visuel clair et intuitif.

1.5.1.1.2 Utilité maquettes

Ces maquettes ont permis, de se faire une idée globale du site avant de commencer le développement, définir la navigation entre les différentes pages ainsi qu'anticiper l'expérience utilisateur.



1.5.2 Analyse fonctionnelle

Un visiteur pourra créer un compte et se connecter afin d'accéder au site web.
L'authentification permettra d'identifier l'utilisateur, sans ça, il n'aura pas accès aux fonctionnalités du site.

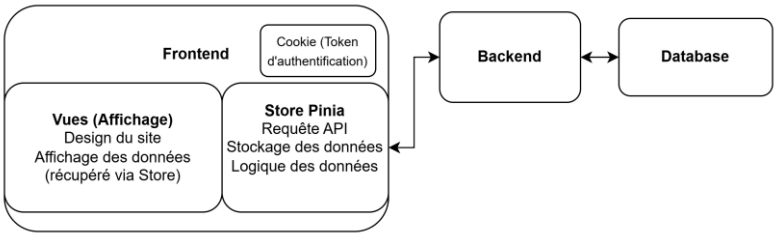
Une fois connecté, l'utilisateur pourra consulter les livres disponibles.
Selon son rôle ou ses droits, il pourra également créer, modifier ou supprimer ertains livres.

L'objectif principal sera de proposer un système de libraire, où les utilisateurs pourront emprunter des livres, les commenter. Tout ça par rapport à un grand catalogue où des filtres et des tris seront disponible pour les aiguiller lors des recherches
Toutes les actions effectuées respecteront les règles de sécurité et de contrôle d'accès définies.

Ainsi, l'application répondra au besoin de gestion structurée des informations tout en garantissant la confidentialité et l'intégrité des données.

1.5.2.1 Schéma fonctionnel

Le schéma fonctionnel présente l'organisation globale des principales parties de l'application ainsi que les flux d'informations entre elles. Il permet de visualiser le fonctionnement général du système sans entrer dans les détails techniques d'implémentation.

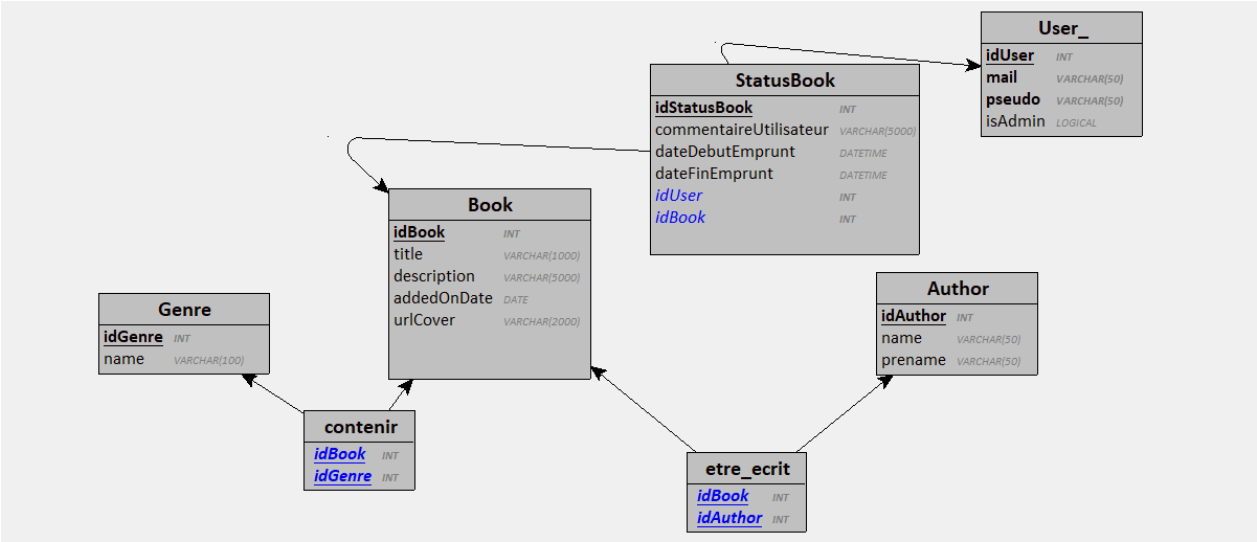


1.5.2.2 Database

La base de données vise à répondre à ces attentes :

1. Un livre peut être relié à plusieurs genres
2. Un genre peut être relié à plusieurs livres
3. Un livre peut être relié à plusieurs auteurs
4. Un auteur peut être relié à plusieurs livres
5. Un livre peut être lié à plusieurs « StatusBook » (Chaque emprunts d'un utilisateur crée un Statut lié au livre)
6. Un « StatusBook » est lié à un seul livre
7. Un utilisateur peut être lié à plusieurs « StatusBook » (Un utilisateur peut faire plusieurs emprunts)
8. Un « StatusBook » est lié à un seul utilisateur

1.5.2.2.1 Modèle logique de données



1.5.3 Choix techniques et alternatives évaluées

1.5.3.1 ExpressJS [Backend]

ExpressJS	AdonisJS (alternative évalué)
Déjà utilisé	Jamais utilisé
Basique	Outils intégrés pratique
Flexible	Environnement cohérent

1.5.3.2 Vue 3 (Composition API) [Frontend]

VueJS
Déjà utilisé
Monté en compétence rapide
Flexible
Affichage réactif
Architecture modulaire

1.5.3.3 Autres outils utilisés [FrontEnd]

TailwindCSS	Développement rapide Acquisition de nouvelle connaissance (personnel)
Axios	Permet de d'utiliser des API simplement Déjà utilisé en cours. (Souvenir personnel positif)
Pinia	Permet l'utilisation de Store (Stockage, requête API centralisé, logique centralisé)
Vue Router	Pour la navigation entre les pages (Permet de gérer les autorisations, et de transférer des données d'une vue à une autre)

1.5.3.4 Autres outils utilisés [BackEnd]

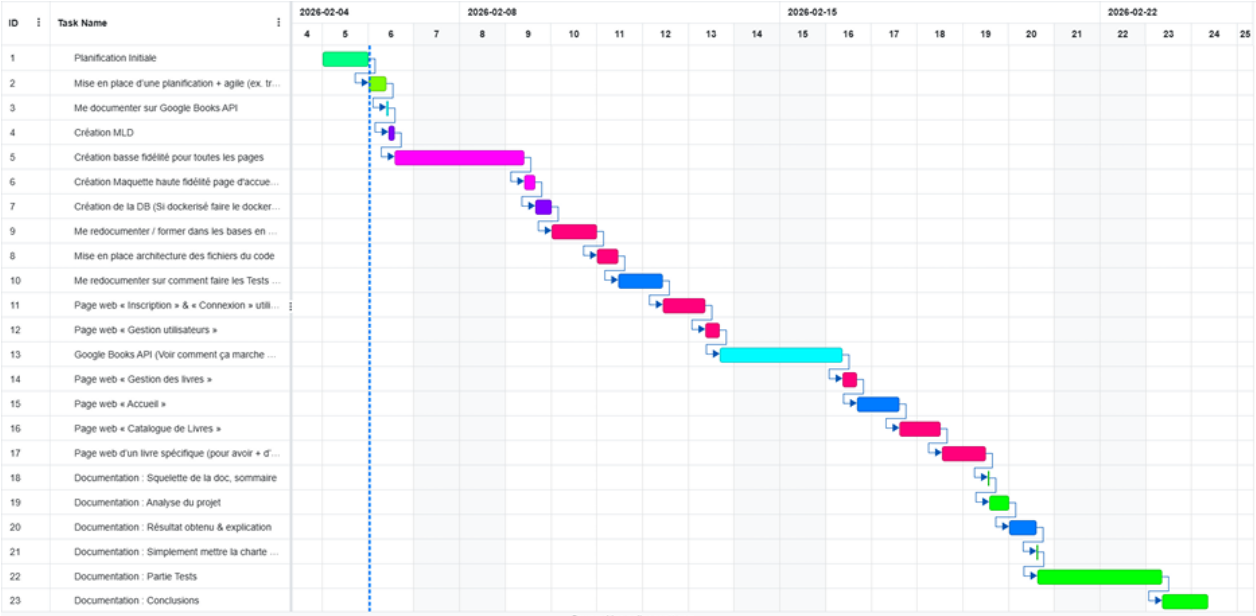
Sequelize	Permet des interactions avec la DB efficace et userfriendly (Requête CRUD, création modèles de données etc..) Déjà utilisé en cours. (Souvenir personnel positif)
Docker	Container (MySQL et phpmyadmin), permet une utilisation de DB très simple Déjà utilisé en cours. (Souvenir personnel positif)
CookieParser	Permet l'utilisation très simplement des cookies (Pour gérer l'authentification par exemple)

1.5.4 Sécurité : authentification, autorisations, gestion des secrets

La sécurité de l'application repose sur l'externalisation des données sensibles via des variables d'environnement (. env) afin de protéger les secrets (clé JWT, accès base de données). L'authentification à l'API est assurée par un middleware (auth) vérifiant le token JWT, tandis qu'un middleware (authorize) d'autorisation contrôle les droits d'accès aux ressources. La configuration CORS limite les requêtes aux origines autorisées, renforçant ainsi la protection contre les appels non légitimes depuis le navigateur.

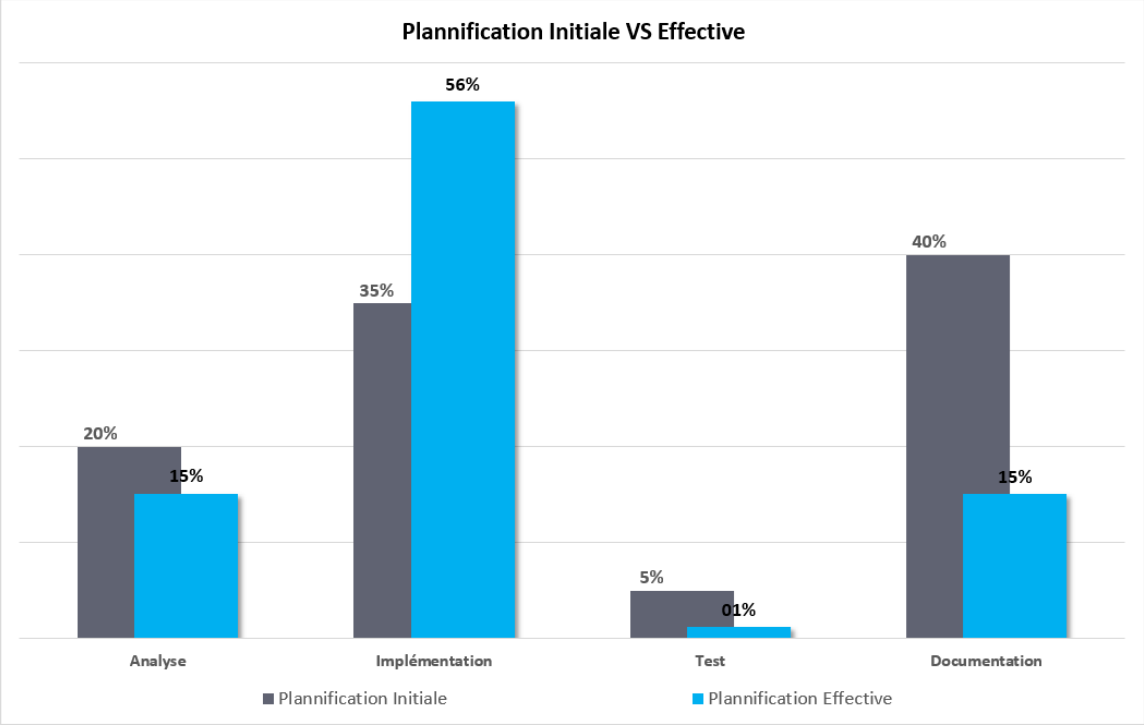
1.6 Planification et suivi

1.6.1 Planification initiale (jalons, Gantt, tâches, date de début / fin, Milestone)



1.6.2 Planification révisée et écarts (explications causes + actions)

Je n'étais plus bien habituée aux projets de l'ETML donc ma gestion a été assez mauvaise, aussi j'ai favorisé l'implémentation étant donné que je préférais monter en compétence technique plutôt qu'en documentation, cependant je regrette un peu d'avoir tardé à effectuer la documentation.



Le « Journal de travail » est partagé dans Annexes.

1.7 Réalisation

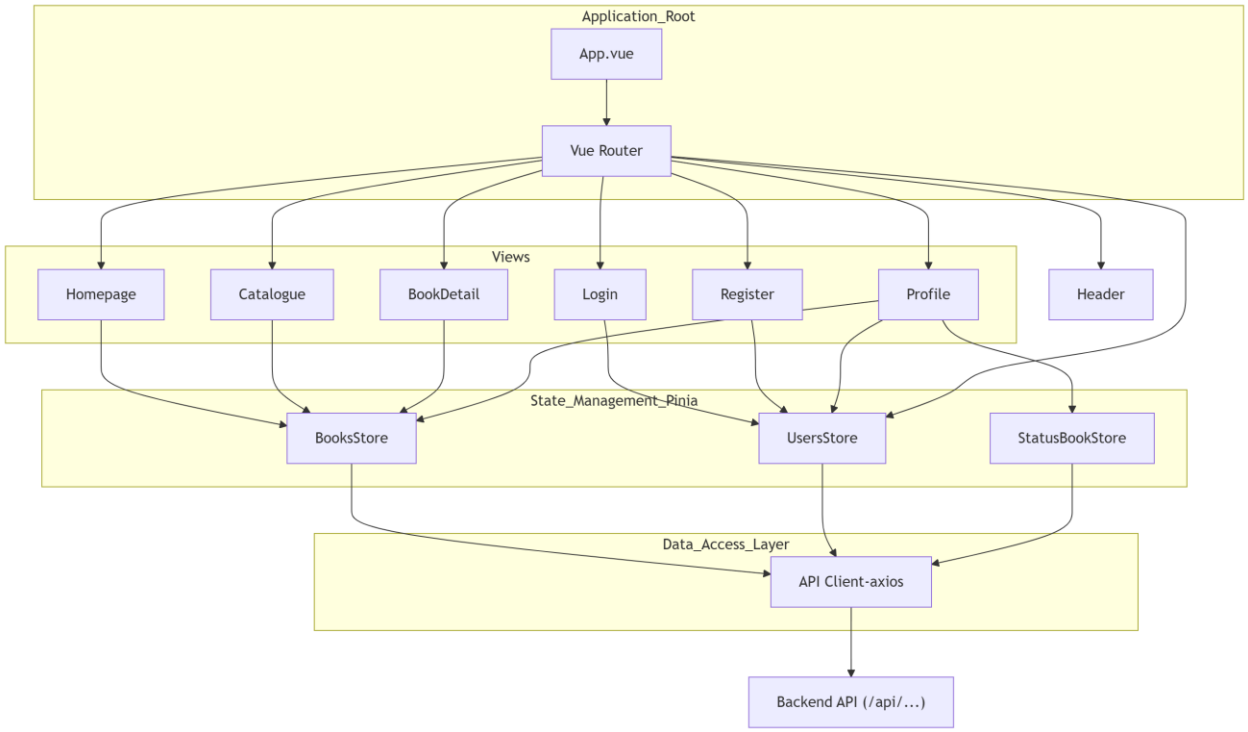
1.7.1 Description de l'implémentation (composants, modules, conventions)

Dans l'état du projet actuel, le site web n'est toujours qu'en développement, il n'a pas de mise en production, ni d'environnement de test à proprement parler, les tests sont effectués dans l'environnement de développement.

1.7.1.1 Architecture technique de la solution

1.7.1.2 Schéma technique Frontend

Le 1 Schéma technique frontend permet une visualisation globale du frontend et de comment il est construit, quelles données chaque vue permet d'afficher, et plus directement, quelles Vues sont reliés à quels Stores.



1 Schéma technique frontend

Il y a certains points intéressants à aborder en rapport avec cette architecture, qui peuvent être important d'expliquer, des points positifs comme négatifs.

1.7.1.2.1 Points positifs

1.7.1.2.1.1 Stores

Les stores permettent de gérer en un point les données que l'on souhaite afficher, centralisant le stockage des données, la logique permettant de les manipuler ainsi que la forme dont on les retourne.

1.7.1.2.2 Points négatifs (A corriger)

Voilà certains défauts de mon application que j'ai remarqué après-coup, que je devrai corriger si c'était un vrai projet mis en prod, mais étant donnée la nature de ce projet, et le manque de temps, je pense que simplement expliquer les problèmes suffit, voir est encore plus utile que les corriger.

1.7.1.2.2.1 Chargement des informations des livres [A optimiser]

Les diverses requêtes API dans le frontend, pourrait être optimiser afin de simplifier le code et gagner du temps d'exécution, si le site devait prendre une charge supérieur de données (livres, commentaires, user, etc…) une refonte de la logique serait inévitable pour éviter de surcharger le site.

```
async fetchBooks(syncAll) {
      this.loading = true;
      this.error = "";
      try {
        const res = await api.get("/api/books");
        this.books = res.data.data ?? res.data;
        if (syncAll == true) {
          await this.syncPopularity();
          await this.syncStatus();
          await this.calcAvailability();
          await this.syncGenres();
          await this.syncAuthors();
        }
```

1.7.1.2.2.2 Liaison Vue Router – UsersStore [Négatif]

Cette connexion se fait via lu Router Guard ci-dessous, qui va utiliser l'action FetchMe du UsersStore, qui va récupérer à chaque changement de routes les informations relatives à l'utilisateur connecté, permettant d'accorder ou non l'accès à la vue en question, jusqu'à maintenant il n'y a pas vraiment de mal, ce n'est pas de la sécurité mais un confort pour les utilisateurs, s'ils sont déconnecté cela les redirige directement vers la page de login. Cependant le problème viens du FetchMe qui va aussi stocker dans le localStorage les informations de l'utilisateur, comme par-exemple récupérer l'id de l'utilisateur, permettant par exemple dans la page de profile d'afficher les livres empruntés par rapport à ce userId.

```
router.beforeEach(async (to, from, next) => {
  const userStore = useUsersStore();
  if (to.meta.requiresAuth) {
    try {
      if (!userStore.idUser) {
        await userStore.FetchMe();
      }
      next();
    } catch (error) {
      next("/login");
    }
  } else {
    next();
  }
});
```

1.7.1.2.2.2.1 Le problème

Ce n'est une mauvaise chose en soit d'utiliser le localStorage, le vrai problème vient de ce qui m'a poussé à utiliser ce FetchMe ici, à savoir la page de Profile, qui va utiliser les valeurs stocké dans le store comme params (paramètre) d'une route de l'API « GetBooksByUserId », que j'utilise ici pour récupérer le livre de l'utilisateur connecté.

Mais un utilisateur malveillant pourrait modifier le paramètre de la requête, ne mettant pas le sien mais celui de quelqu'un d'autre, et pourrait donc accéder aux données privés de quelqu'un d'autre.

C'est donc le backend qui est mal pensé et dangereux :

```
export async function getBooksByUserId(req, res) {
    const { id } = req.params;
    const books = await models.StatusBook.findAll({
        where: { idUser: id },
        include: [{ model: models.Book, as: 'idBook_Book' }]
```

1.7.1.2.2.2.2 Solution possible

Une meilleure manière de faire serait que le backend récupére l'id via req.user.idUser (qui est défini de toute façon via le middleware auth), ainsi l'utilisateur connecté pourra seulement récupérer la liste de ses emprunts.

1.7.1.2.2.2.3 Pourquoi ce problème est survenu

Lorsque je faisais mon backend au début du projet, j'avais imaginé que cette route permettrait d'à la fois récupérer les emprunts d'un utilisateurs connectés (comme je le fais actuellement, mais je ne voyais pas encore le problème), et pour l'interface admin de pouvoir afficher facilement les emprunts d'un autre utilisateur, mais en réalité il faudrait créer 2 routes, dont l'une nécessitant des droits admin pour être utilisé.

1.7.1.3 Configuration des systèmes et services

1.7.1.3.1 CORS

Permet de définir les droits concernant l'utilisation de l'API, acceptant pour notre site, les credentials et cookies ainsi que les requêtes CRUD.

```
app.use(
  cors({
    origin: "http://localhost:5173", // PAS '*'
    credentials: true,               // autorise cookies / credentials
    methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allowedHeaders: ["Content-Type", "Authorization"],
  })
);
```

1.7.1.3.2 Cookie-Parser

Permet de récupérer les cookies lors d'une requête http via req.cookies.

```
app.use(cookieParser());
```

1.7.1.3.3 Express

Permet de lire le json dans le body

```
app.use(express.json());
```

1.7.1.4 Gestion des accès et de la sécurité

Les permissions et la sécurité sont évidemment gérer dans le backend, via 2 middlewares, « auth.js » qui vérifie que le client est bien un utilisateur correctement authentifié et « authorize.js » qui va accorder ou pas l'accès à certaines routes selon les permissions attribué à l'utilisateur (admin ou simple utilisateur).

1.7.1.4.1 auth.js

Le middleware d'authentification « Auth », il sert à valider que le client qui fait une requête à une route protégé de l'API soit bien un utilisateur authentifié, l'authentification se fait via un token stocké dans un cookie sur le navigateur du client.
Pour que le client reçoive ce token, la route de login qui est utilisé lors de la connexion d'un utilisateur, envoie « access_token » contenant le token, que le navigateur va ensuite stocker.

Pour pouvoir utiliser et récupérer des cookies dans l'API nous avons eu besoin d'utiliser le middleware « cookie-parser », configurer de manière à rendre accessible les cookies via l'objet req.cookies.

1.7.1.4.2 authorize.js

Lors de l'exécution du middleware « auth » (qui est antérieur à l'exécution du middleware authorize), il va définir les valeurs de l'objet « req.user » selon les valeurs présentes dans le token.
Le middleware authorize vérifie via la valeur de « req.user.isAdmin » si l'utilisateur possède les droits admin ou non, et accorde l'accès à la route à l'utilisateur si ses permissions sont suffisante pour utiliser la route en question.

1.8 Tests et validation

1.8.1 Tests fonctionnels

Dans la mise en place d'un petit projet comme celui-ci, nous avons estimé que les tests les plus pertinents étaient les tests fonctionnels car ils permettent de garantir un résultat final correcte sans prendre trop de temps à être mis en place.

Fonctionnalité	**Action utilisateur (***italique beige = utilisateur pas authentifié***)**	**Résultat attendu**	**Résultat obtenu**

Login valide	*Saisir email + mot de passe correct*	*Authentification réussie, redirection vers homepage*	OK
Login invalide	*Saisir identifiants incorrects*	*Message d'erreur affiché, pas de redirection*	OK
Champs vides	*Cliquer sur login sans remplir*	*Message d'erreur disant qu'identifiant et mdp sont nécessaires, aucune requête envoyée*	OK
Inscription	*Remplir email + pseudo + mot de passe valides et soumettre*	*Compte créé avec succès et possibilité de se connecter*	OK
Inscription	*Utiliser un email déjà existant*	*Message d'erreur indiquant que l'email est déjà utilisé*	NOK
Refresh page	Rafraîchir la page connecté	L'utilisateur reste connecté	OK
Logout	Cliquer sur logout	Token supprimé, redirection vers login	OK
Liste livres	Aller sur page livres	Liste des livres affichée correctement	OK
Détail livre	Cliquer sur un livre	Informations détaillées affichées	OK
Livre inexistant	Accéder à un ID invalide	Message d'erreur ou 404 affiché	OK
Voir emprunts	Aller sur profil	Liste des livres empruntés affichée	OK
Aucun emprunt	Profil sans livres	Message "Aucun livre emprunté"	OK
Emprunter livre	Cliquer sur "Emprunter"	Livre ajouté aux emprunts	OK
Retour livre	Cliquer sur "Rendre"	Livre retiré des emprunts	NOK
API sans token	Requête vers route protégée sans auth	Réponse 401 Unauthorized	OK
Token expiré	*Utiliser token expiré*	*Redirection vers login*	OK
Double clic	Cliquer plusieurs fois sur "Emprunter"	Un seul emprunt créé	OK
Serveur indisponible	API hors ligne	Message d'erreur affiché	NOK
Homepage	Etant sur la page homepage	Je vois les 5 livres les plus populaires	OK
Homepage	Etant sur la page homepage	Je vois les 5 livres les plus récemment ajoutés	OK
Homepage	Vérifier l'affichage des informations d'un livre	Chaque livre affiche titre, couverture, auteur, genre, popularité et statut	NOK
Catalogue	Etant sur la page catalogue (sans filtre activé)	Je peux voir tous les livres	OK
Catalogue	Lorsque que j'utilise un filtre (auteurs/genres/disponibilité)	L'affichage des livres s'adapte n'affichant plus que les livres correspondant.	OK
Catalogue	Si plusieurs filtres sont activés	Cela affiche les livres qui réponde au moins à 1 filtre	OK

Catalogue	Si + de 10 livres sont affiché	Un système de pagination apparait	NOK
Détails livre	Partout dans le site, si une prévisualisation d'un livre est disponible, lorsqu'on clique dessus	On arrive sur la page de détails du livre en question	OK
Détails livre	Etant sur la page de détails d'un livre	Je vois : cover, préfaces, auteur(s), genre(s), popularité, disponibilité	OK
Détails livre	Etant sur la page de détail d'un livre (disponible) je clique sur « Emprunter ».	Son statut change devenant indisponible, et il est ajouté à ma liste d'emprunts.	OK
Détails livre	Sur la page de détail d'un livre (indisponible)	Il n'est pas possible de l'emprunter	OK
Popularité	A chaque nouvelle emprunt d'un livre	Sa popularité augmente de 1	OK
Profile	Cliquer sur « Se déconnecter »	Me déconnecte du site et me redirige vers login	OK
Profile	Etant sur la page de profile je vois	Mon pseudonyme et email de mon compte	OK
Profile	Etant sur la page de profile je vois	La liste de mes emprunts (titre livre, date fin d'emprunt, et commentaire personnel (qui est vide par défaut mais que je peux remplir)	OK
Profile	Etant sur la page de profile, je vois mes emprunts.	En cliquant sur le bouton « Rendre le livre » le statut du livre change devenant disponible et la date de rendu du livre s'actualise	NOK
Historique	Depuis le profil, consulter l'historique complet des emprunts	Les anciens emprunts sont visibles même après retour des livres	OK
Admin - Login	Lorsque je me connecte avec un compte admin	Je peux accéder à l'interface admin	NOK
Admin – Livres	Sur la page d'administration des livres	Je peux voir tous les livres (les filtres auteurs/genres/disponibilité sont disponible)	NOK
Admin – Livres	Si un livre n'est pas emprunté actuellement, et que je clique sur supprimer	Cela le supprime du catalogue (de la db en général)	NOK
Admin – Livres	Si un livre est emprunté actuellement.	Je ne peux pas le supprimer.	NOK
Admin – Livres	Si j'entre le nom d'un livre dans un placeholder « ajouter un livre » et que je clique sur ajouter	Le placeholder se vide et le livre est ajouté au catalogue (avec les infos google books api).	NOK

UI	Toutes les pages du site	Sont responsive sur tablette (écran + petit qu'un PC)	NOK
Catalogue	Utiliser filtre par popularité	Les livres sont **triés** selon leur niveau de popularité	OK
Admin – Utilisateurs	Accéder à la gestion des utilisateurs	Liste complète des utilisateurs affichée	NOK
Admin – Utilisateurs	Bloquer un utilisateur	L'utilisateur bloqué ne peut plus se connecter	NOK
Admin – Utilisateurs	Modifier les informations d'un utilisateur	Les modifications sont enregistrées en base	NOK
Admin – Utilisateurs	Supprimer un utilisateur	L'utilisateur est supprimé de la base de données	NOK
Admin – Emprunts	Accéder à la liste des emprunts en cours	Tous les utilisateurs ayant un emprunt actif sont affichés	NOK
Admin - Sécurité	Accéder à l'interface admin avec un compte non-admin	Accès refusé	NOK
Persistance	Redémarrer le site/web/db	Les données du site sont toujours présentes.	OK
Deploiement	Le site a bien été déployé en production	En allant sur l'herbergement de test w3.pm2etml.ch mise à disposition par le chef de projet, je peux accéder au site	NOK
Test unitaire backend	Toutes les routes du backend sont testé via des tests unitaire.	Si je fais des changements cassant l'API volontairement que j'exécute les tests unitaires, les tests ne passent plus à cause des nouvelles erreurs mise exprès.	NOK
Test unitaire backend	Toutes les routes du backend sont testé via des tests unitaire.	En temps normal, quand mon backend est fonctionnel, tout les tests unitaires réussissent	NOK

1.8.1.1 Explication des résultats

- Inscription email déjà utilisé [Rapide à régler]

L'inscription ne fonctionne pas, cependant j'ai oublié de le préciser dans un message d'erreur.

- Rendre un livre [Pas demandé CDC]

C'est une fonctionnalité logique et pas très compliquée à implémenter, mais j'avais d'autre priorité qui passaient avant, en plus ce n'était pas demandé dans le cahier des charges.

- Afficher un message lorsque l'API est down

Fonctionnalité que j'implémenterais clairement si je continuais le projet, cependant cela ne faisait pas partie de ma liste des priorités, une fois que j'ai remarqué que j'avais oublié, je me suis dis que je le ferai à la fin si il me restait du temps.

- Dans la homepage chaque livre est affiché avec les informations suivantes : auteurs, genres, popularité, disponibilité, etc…

Cette fonctionnalité par-contre j'ai fais exprès que ce ne soit pas le cas, je pense que ce n'est pas utile, et que cela chargerait la page pour une utilité moindre, j'ai préféré faire en sorte que peu importe ou dans le site, si un livre est affiché, en cliquant dessus on est redirigé vers sa page de détails, où l'on peut trouver toutes les informations le concernant. Je trouve cette solution meilleure

- Dans le catalogue, si plus de 10 livres sont affiché un système de pagination apparait

Pour ce point aussi j'ai sciemment fait d'une autre manière que je trouve plus propre, je pense que la page web a la place pour afficher 12 livres sans que ce soit dérangeant, le système de pagination par-contre apparait même si il n'y en a pas besoin (si seulement 4 livres sont affiché par-exemple), j'améliorerai ce point si je continuais le projet, car c'est très rapide à faire, cependant ça reste un défaut minime, c'est pour ça que pour l'instant je l'ai laissé.

- Rendre un livre sur la page de profile [Pas demandé CDC]

Priorité moindre et pas demandé dans le CDC.

- UX responsive

Ce n'était pas ma priorité, il l'est un peu grâce à tailwind qui s'en occupe en partie tout seul, mais c'est très loin d'être suffisant, ce serait une de mes priorités si je continuais le projet

- Les tests admins (Tous en général)

Le site est prêt à implémenter son interface admin (la route auth renvoie l'id et le isAdmin dans req.user, etc) il n'y a donc pas énormément de changement à faire dans l'architecture pour accueillir un système d'administrateur (le backend a déjà le middleware authorize qui s'en occupe, et certaines routes sont nécessite déjà les droits administrateurs (pour supprimer un livre par exemple), cependant le frontend n'est pas du tout commencé pour les administrateurs.

Donc en résumé si je continuais le projet, j'aurais de bonne base pour commencer cette partie, mais rien n'est commencé pour l'instant.

- Déploiement

Le site est resté en local sur ma machine, il n'y a pas eu de déploiement (Manque de temps)

- Tests unitaires

Aucun test unitaire n'a été mis en place, par manque de temps (et honnêtement je préférai avancer sur le site que faire des tests unitaires pour les routes)

1.8.2 Test unitaires (Pas mis en place)

Si j'avais eu plus de temps des tests unitaires auraient aussi été les bienvenues, permettant de garantir l'intégrité du site même après avoir effectué des modifications.

1.9 Conclusion et bilan personnel

1.9.1 Objectifs atteints / non atteints (preuve)

Objectivement, ils manquent pas mal d'aspect demandées dans le cahier des charges voir Tests et validation, la partie de gestion par les administrateurs du site n'est pas implémenté, le site n'est pas déployé en production, il n'y a pas de tests unitaires, etc.
Par-contre la partie principale du site, à savoir sa capacité à gérer des utilisateurs qui se créent des comptes, se connecte, pour ensuite accéder à l'affichage des livres et aux fonctionnalités qui suivent (emprunt de livre, filtre et tri du catalogue) ainsi qu'une majeure partie du backend est tout à fait utilisable.

1.9.2 Points positifs / négatifs

D'un point de vue plus subjectif, ce projet m'a bien remis sur les rails pour la suite des évènements, comme mentionné dans les Risques organisationnels cela faisait un an que je n'avais plus codé car j'étais dans un stage (Infra/Réseau), et je n'étais plus habitué aux projets et façons de travailler de l'ETML, le début a été lent et compliqué, je ne me sentais plus prêt du tout pour le TPI.

Mais arrivé au bout de ce P_Appro1, je me sens beaucoup mieux vis-à-vis de la suite, j'ai pu reprendre mes marques, même si ça ne se voit pas forcément dans ce projet, car ce n'étais pas le cas au début.

1.9.3 Difficultés rencontrées et solutions

Les difficultés rencontrées ont surtout concerné tout ce qui était analyse au début du projet, pour l'analyse initial en grande partie, malheureusement je n'ai « compris » que trop tard qu'une des meilleures manières de faire était d'écrire honnêtement toutes mes idées, et que cela n'était pas grave si le résultat ne suivait pas.

J'ai finalement écrit les tests fonctionnels par-exemple quelque jours avant le rendu, en écrivant des tests que je savais pertinemment que je ne pouvais pas mettre en place par manque de temps.
C'est bête car c'était une des seuls choses qui me faisait peur lors de la planification initiale, que je n'ai finalement pas éviter, en résumé les j'ai subis seulement les effets négatifs de l'analyse initiale sans en tirer les effets positifs. Tout ça parce que je me suis compliqué la tâche pour rien.

Finaliser les tests fonctionnels, même à quelques jours de la fin du projet m'a réellement allégé, ça a été en soit ma solution, qui n'aurait pas eu lieu d'exister si j'avais fait les choses simplement. Pour les prochains projets je partirai avec cette certitude de plus.

1.9.4 Apprentissages et perspectives d'amélioration

Ce que je regrette dans ce projet, c'est de ne pas m'être lancé directement à fond au début, par peur de l'incertain au début j'étais frileux à me lancer dans certaines tâches, car je n'étais pas sûr de bien comprendre etc. Ce qui m'a valu de prendre pas mal de retard au début, sans ne rien m'apprendre.

Maintenant arrivé à la fin du projet, je remarqué que j'ai appris bien plus choses dans la deuxième moitié du temps de projet, étant donné que par soucis de temps j'étais obliger de me lancer, et ça n'a pas si mal fonctionné.

Dans le futur, si je suis bloqué je tâcherai de fournir un effort pour me lancer, sachant que le début est souvent le plus difficile.

2 Partie 2 - Documentation du produit

2.1 Installation du projet (environnement développement)

Le seul environnement actuellement sur lequel le site a été mis en place est un environnement de développement, un simple poste Windows 10, suivant les spécifications ci-dessous.

2.1.1 Environnement de base et applications de base requis

Sur un OS **Windows 10**, installer les applications suivantes :

- Node.JS (v24.x)
- Docker Desktop
- Outils de développement, ex. Visual Studio Code (Si vous voulez apportez des modifications)
- GitHub (Facultatif)

2.1.2 Installation du projet

1. Installation du projet
   (Soit via un git clone avec l'url : https://github.com/JulienETML-hub/P_Appro1_Bibliotheque)
   (Soit en téléchargeant et dézippant directement le zip en annexe A AJOUTER LIEN VERS ZIP ET/OU SON NOM)
2. Dans le dossier P_Appro1_Bibliotheque\code\Noveling_Backend :
   Créer un fichier .env (En activant les extensions de noms de fichiers dans l'explorer windows)
   Remplir le fichier comme ci-dessous en
   Modifiant la valeur de « GOOGLE_BOOKS_API_KEY », en mettant votre propre Google Books API KEY que vous pouvez obtenir en suivant la documentation suivante : Lien vers la documentation Google Books APIs
   Modifier la valeur de « JWT_SECRET », vous pouvez mettre n'importe quelle valeur, mais dans un environnement de dévelopemment cette valeur doit rester complétement secrète.

```
PORT=3000
GOOGLE_BOOKS_API_KEY="Exemple123"
JWT_SECRET="Exemple123"
NODE_ENV="dev"
```

3. A la racine du projet dans l'invite de commande exécuter :
   « cd docker »
   « Docker compose up -d »
   Les Conteneurs docker avec la DB se crée.
4. A la racine du projet dans l'invite de commande exécuter :
   « cd code/Noveling _backend »
   « npm install »
   « npm run dev »
5. A la racine du projet dans l'invite de commande exécuter :
   « cd code/Noveling _frontend »
   « npm install »

« npm run dev »

2.2 Guide d'utilisation

Une fois la partie Installation du projet (environnement développement) effectué, simplement se rendre via un navigateur (Chrome, edge, firefox), sur l'adresse local « Localhost :5173 » et vous arriverez sur le site, l'utilisation du site devrait être intuitive, je ne le détaillerai donc pas dans ce rapport.

2.3 Architecture technique (rappel)

2.3.1 Schéma d'architecture

Voir : Schéma fonctionnel

Voir : Schéma technique Frontend

Voir : Modèle logique de données

3 Résumé

3.1 Contexte et mandat

Le mandat consistait à concevoir et développer une application web complète permettant la gestion et l'emprunt de livres en ligne, l'objectif principal était de mettre en place un frontend ainsi qu'un backend sécurisé, allant récupérer des informations dans une base de données.

Le projet devait inclure une analyse préalable, une planification structurée, une conception technique (modèle de données, maquettes UI), ainsi qu'une implémentation fonctionnelle propre.

Une fois l'application web développé elle devait être testé, selon des tests fonctionnels prédéfinis ainsi que des tests unitaires pour le backend.
Et finalement elle devait être déployé sur un environnement mis à disposition par l'ETML.

3.2 Solution réalisée

Finalement ce qui a été réalisé consiste en : Une API rest sécurisé, permettant d'aller chercher des informations dans une base de données virtualiser via conteneur docker.
Un Frontend en VueJS qui permet d'afficher les données récupérées via le backend, répondant aux demandes du cahier des charges, hormis l'interface administrateur.
Tout cela accompagné de maquette, MLD, planification initial, ainsi qu'un rapport décrivant l'ensemble du projet.

3.3 Résultats principaux

Finalement, l'applications permet aux utilisateurs de se créer un compte, se connecter de manière sécurisée et emprunter des livres. La gestion des rôles et la protection des routes garantissent un accès contrôlé aux fonctionnalités(backend).

Les séparations des diverses parties du site permettent une compréhension facile, et des maintenances et améliorations pas compliqué à effectuer.

Malheureusement les tests ainsi que le déploiement n'ont pas été fait, ce qui rend le rendu incomplet, par rapport aux demandes du cahier des charges.

3.4 Limites et suites

Le projet ne se heurte pas à des limites de conception, ce qui est un très bon point si on suite était envisagé, pour l'instant la suite logique serait de s'occuper des parties manquantes du cahier des charges (tests unitaires, déploiement, interface administrateurs).
Une fois cela fait des idées d'amélioration pourrait être l'implémentation d'un CI/CD qui testerait le site à chaque déploiement, permettant d'assurer son bon fonctionnement.

4 Annexes

A. Journal de travail

Le journal de travail au format excel est disponible en annexe

04 févr	2 h 00 min	Analyse	Création CDC (P_Fisherman) + discussion avec M. Meylan (première discussion concernant le P_Appro)
04 févr	1 h 00 min	Absent	
04 févr	45 min	Analyse	Début d'analyse sur comment / avec quoi faire le projet fisherman
04 févr	45 min	Analyse	Discussion avec M. Meylan concernant le projet à faire
04 févr	45 min	Analyse	Choix de thème proposé par M. Meylan (P_Biblioèthque)
04 févr	45 min	Documentation	Création CDC avec le contenu du CDC donnée par M. Meylan avec la forme du nouveau CDC
05 févr	45 min	Analyse	Début de mise en place
05 févr	1 h 30 min	Analyse	Analyse initiale
05 févr	45 min	Analyse	Documentation sur Google Books API
05 févr	2 h 00 min	Analyse	Planification & analyse initiale
05 févr	20 min	Absent	Discussion avec des profs pour le TPI (pas pour ce projet)
05 févr	25 min	Analyse	Discussion concernant les technos avec M. Meylan
05 févr	45 min	Analyse	Création MLD (à finaliser)
05 févr	1 h 00 min	Analyse	Tâche diverse (création github, remplir jnvtrl, création word rapport etc.)
06 févr	45 min	Analyse	MLD MCD Fini
06 févr	25 min	Absent	Discussion avec un prof pr TPI
06 févr	10 min	Absent	Discussion M. Duding
06 févr	15 min	Absent	Conception d'une image pour le design de mon site avec chatGPT
06 févr	45 min	Analyse	Diagramme de gant pour planification (et qqchangement dans la planif initiale)
06 févr	4 h 30 min	Implémentation	Création Maquette pour le site
09 févr	3 h 00 min	Implémentation	Création Maquette pour le site
09 févr	1 h 50 min	Implémentation	Création Maquette pour le site
09 févr	25 min	Implémentation	Création docker compose et script sql
11 févr	45 min	Implémentation	Création docker compose (mysql & phpmyadmin) et db dedans

11 févr	15 min	Absent	Discussion avec un prof pr TPI
11 févr	2 h 00 min	Analyse	Mise en place et début de réapprentissage de vue js
11 févr	2 h 15 min	Implémentation	J'ai eu des problèmes pour mettre en place mon projet vue en utilisant tailwind (car j'utilisais une version ancienne qui avait des changements dans l'utilisation)
11 févr	45 min	Implémentation	Début page de connexion (surtout pour me remettre dans le html/css pour l'instant et tailwind
12 févr	3 h 45 min	Implémentation	Page de connexion (html/css seulement) fini, commencement d'analyse des routes dont j'ai besoin, à continuer cette après-midi.
12 févr	2 h 30 min	Implémentation	Documentation sur backend (avec express), et google API books
12 févr	30 min	Analyse	Analyse conceptuelle des routes Api à faire
13 févr	45 min	Implémentation	Fix ma db et docker qui avait des prbl
13 févr	1 h 00 min	Analyse	Analyse route API a faire et relecture du cdc
13 févr	1 h 15 min	Implémentation	Mise en place API Rest
13 févr	3 h 45 min	Implémentation	Création des routes simple (get livre) et create livre pas encore fini (il faut s'occuper du create mais aussi d'aller chercher un livre via l'API de google books)
23 févr	20 min	Absent	Discussion avec Sofiène
23 févr	4 h 30 min	Implémentation	Route Books et Users de l'API (utilisation de l'API google pour ajouter des livres)
23 févr	1 h 00 min	Implémentation	J'ai essayé de tester les routes login et register de plusieurs manières mais pour l'instant ce n'est pas vraiment au point
24 févr	5 h 15 min	Absent	Malade
25 févr	6 h 45 min	Absent	Malade
26 févr	35 min	Documentation	Regroupement d'informations dans le rapport (sous forme brutes pour l'instant sans aucune mise en page ou autre)
26 févr	4 h 10 min	Implémentation	Page de connexion avec Vue js (se connecter à un user présent dans la db)
26 févr	1 h 00 min	Implémentation	Retourner liste de livre (simplement dans la console pour l'instant) mais sachant que la route qui recupere la liste des livres a besoin d'un authentification ce n'est pas si simple, donc actuellement c'est bon j'ai

			réussis (très très moche et inutilisable mais au moins je sais comment faire)
26 févr	1 h 00 min	Analyse	Discussion avec M. Meylan concernant le projet, il m'a montré comment je pourrais faire pour mieux structurer mon code en découpant et en faisant des "store" le frontend, et aussi en quoi utiliser localstorage peut s'avérer pratique, donc je pense qu'à partir de demain je vais commencer ça
27 févr	3 h 45 min	Implémentation	Implémentation store avec pinia + page de connexion fonctionnel, qui redirige vers homepage une fois qu'on est connecté
27 févr	1 h 00 min	Analyse	J'ai réfléchi à comment je voudrais que mes pages soient gérer, car je n'ai pas réussi à faire comme je le voulais instinctivement (router.push directement dans un componenent, peut-etre uqe j'ai mal fais mais quand j'essayais ma page web crashait complétement)
27 févr	1 h 40 min	Implémentation	Mise en place d'un router pour gérer les différentes pages, parcontre pour l'instant j'ai encore qqproblemes de compréhension mais j'ai limpression qu'une fois que j'aurais compris ce qui me manque ca sera un moyen très simple et efficace pour gérer mes pages.
27 févr	25 min	Absent	je suis parti 30mn plutôt pour rattraper un test
02 mars	5 h 15 min	Implémentation	Toujours les routes et début front-end, je suis entrain de faire en sorte que le store recoivent toutes les données dont jai besoin pour le frontend (pour l'instant je prend tout au chargement de la page meme si c'est pas la meilleure idée en prod c'est juste pour avoir une base simple pour l'instant)
03 mars	6 h 45 min	Implémentation	Page de catalogue en cours, j'affiche tout les livres et demain je ferai les filtres et la page de détails des livres.
05 mars	3 h 45 min	Documentation	Début rédaction rapport au propre

05 mars	2 h 45 min	Implémentation	
09 mars	6 h 45 min	Implémentation	Les filtres sont fais + page détails + page de profile
11 mars	1 h 45 min	Documentation	Rédiger texte fonctionnelles
11 mars	5 h 00 min	Implémentation	Système de pagination + afficher emprunts livre dans page profile + register + séparation page catalogue en component partial (Je l'avais fais mais pas assez donc il faut que je découpe encore, pour pouvoir réutilisr ces composant dans l'interface admin par exemple) + demain ducoup continuer découpage en component + petit + interface admin + rapport de projet + doc mise en place
12 mars	6 h 45 min	Documentation	Rédaction Rapport (Création schéma technique etc), demain finir rapport, faire release TOUT CLEAN + pas oublier tester les test et commenter selon résultat, pq ca marche pas etcetc
13 mars	2 h 30 min	Documentation	Description implémentation
13 mars	1 h 30 min	Test	Tester les divers tests fonctionnels, documenter les résultats dans le rapport et expliquer les résultats
13 mars	2 h 15 min	Documentation	Rédaction rapport (conclusion, comparaison planif + travail effectué + faire livrable + glossaire + résumé)
13 mars	10 min	Absent	Démonstration site M. Meylan

B. Sources et références

4.1 Références

Lien vers GitHub

Lien vers Trello

4.2 Sources

VueJS documentation
W3Schools
TailWindCSS documentation

4.3 Usage IA

Site utilisé : https://mammouth.ai/
LLM utilisé (mammouth en propose plusieurs) : ChatGPT-5.2

Usages :

- Génération image (pour le site web)
- Demande d'aide pour le code (surtout en vuejs, pour la logique, mais pour cette partie là je ne demandais que des parties très ciblés et je devais l'implémenter, ou alors si ce n'était pas le cas je prenais le code qu'il me donnait et l'adaptait selon mon idée de base, sans lui demander de le faire, cela me forçait à comprendre comment ça fonctionnait)
- Génération html css (Au début du projet je faisais totalement mes pages d'html et de css, mais à la fin du projet, trouvant que faire du html css était pas très compliqué mais juste long je lui ai demandées de me donner directement tout le code html css (pour certaines pages), que j'adaptais selon mes besoins, et ça sans lui demander de le faire.
  Cela me faisait gagner du temps, sans me faire perdre trop de connaissance, à mon avis)
- Génération UML (pour le rapport, en lui donnant mon code et lui demandant de générer un code UML, j'ai dû le corriger par contre, car il n'arrivait pas à récupérer tout le code)

C. Liste des livrables remis

Livrable	Emplacement (lien/chemin)	Remarques
Journal de travail	*Journal-de-Travail_MaresJulien_P_Appro1.xlsx*	*À compléter*
Code source du projet	*code*	
Docker-compose & script SQL	*docker*	*Script sql s'exécute dans le conteneur lors de sa création*
MLD looping	*MCD_MLD_P_BibliothèqueDB.loo*	
Guide d'installation	*README.md*	*Affichage disponible sur le repo GitHub*
Planification initiale	*Online Gantt 20260206.pdf*	*Sous forme diagramme de gant*
Release GitHub	*Lien vers Release GitHub*	
Maquettes	*Maquettes*	